PASSWORD CHECKER

Git Collaboration and Continuous Integration — Group Project Report

Team members

Rania Rahoui — Member A, password rule checks and project page

Yasmine Boutchiche — Member B, password strength scoring

Kamelia Magdoline Milha — Member C, weak-pattern detection

Repository: github.com/rania-rahoui/GitLab_Assignment2_PasswordChecker

Date: October 25, 2026

Project objective: Build a small Python password checker while demonstrating branching, commits, pull requests, peer review, merge conflict resolution, automated tests, and GitHub Pages deployment.

1. Project overview

The Password Checker evaluates passwords using basic rule checks, a strength score, and weak-pattern detection. The project was developed collaboratively in a public GitHub repository with separate contributions from the three members. The final repository includes source files, tests, a GitHub Actions workflow, documentation, and a project page.

2. Team responsibilities

Rania Rahoui: implemented basic password rules (minimum length, uppercase, lowercase, digit, and special character), contributed repository documentation and GitHub Pages content, and merged project pull requests.

Yasmine Boutchiche: implemented password strength scoring from 0 to 5 with Weak, Medium, or Strong labels, added scoring tests, responded to review feedback, and resolved the README merge conflict.

Kamelia Magdoline Milha: implemented detection of common passwords, repeated characters, and sequences, added tests, and demonstrated a failing CI run followed by a corrected passing run.

3. Repository organization and integration

The main branch contains **password_checker/** for application logic, **tests/** for automated verification, **.github/workflows/** for continuous integration, **docs/** for the published project page, and configuration files such as **pytest.ini** and **requirements.txt**. The screenshot shows 24 commits, three contributors, and a successful GitHub Pages deployment after PR #7 was merged.

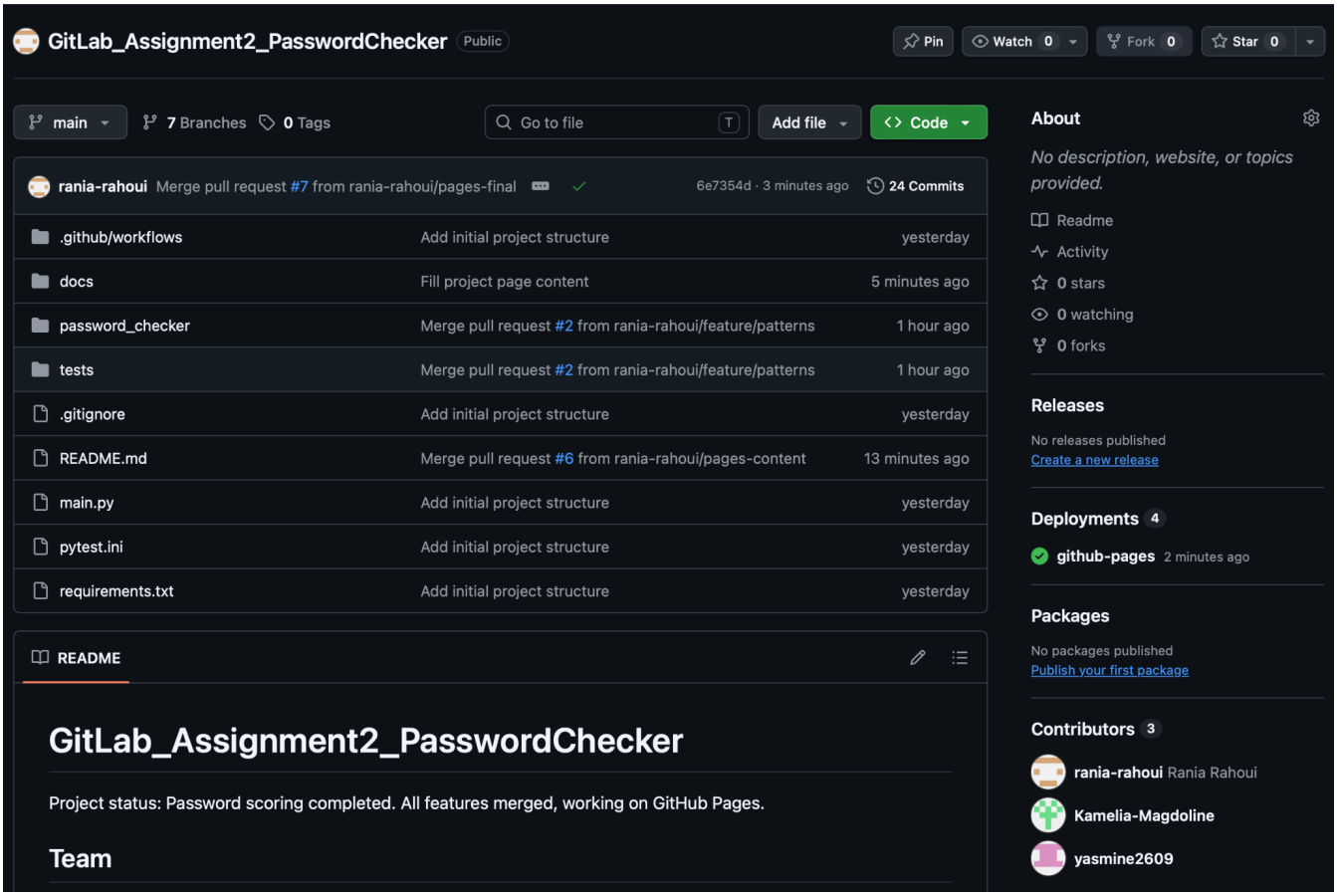


Figure 1. Final public repository on the main branch, with source, tests, documentation, and deployment status.

4. Feature branches and pull requests

Members developed changes in dedicated branches and submitted pull requests for review before integration into main. Yasmine submitted her scoring feature in PR #3. Kamelia submitted the pattern-detection feature in PR #2. Rania submitted project-page updates through PRs #6 and #7. The README status changes were integrated after a deliberate merge-conflict exercise through PR #5.

5. Peer review and review protection

The scoring pull request initially showed two successful automated checks but could not be merged because at least one approving review was required. This demonstrates the separation between automated validation and human review.

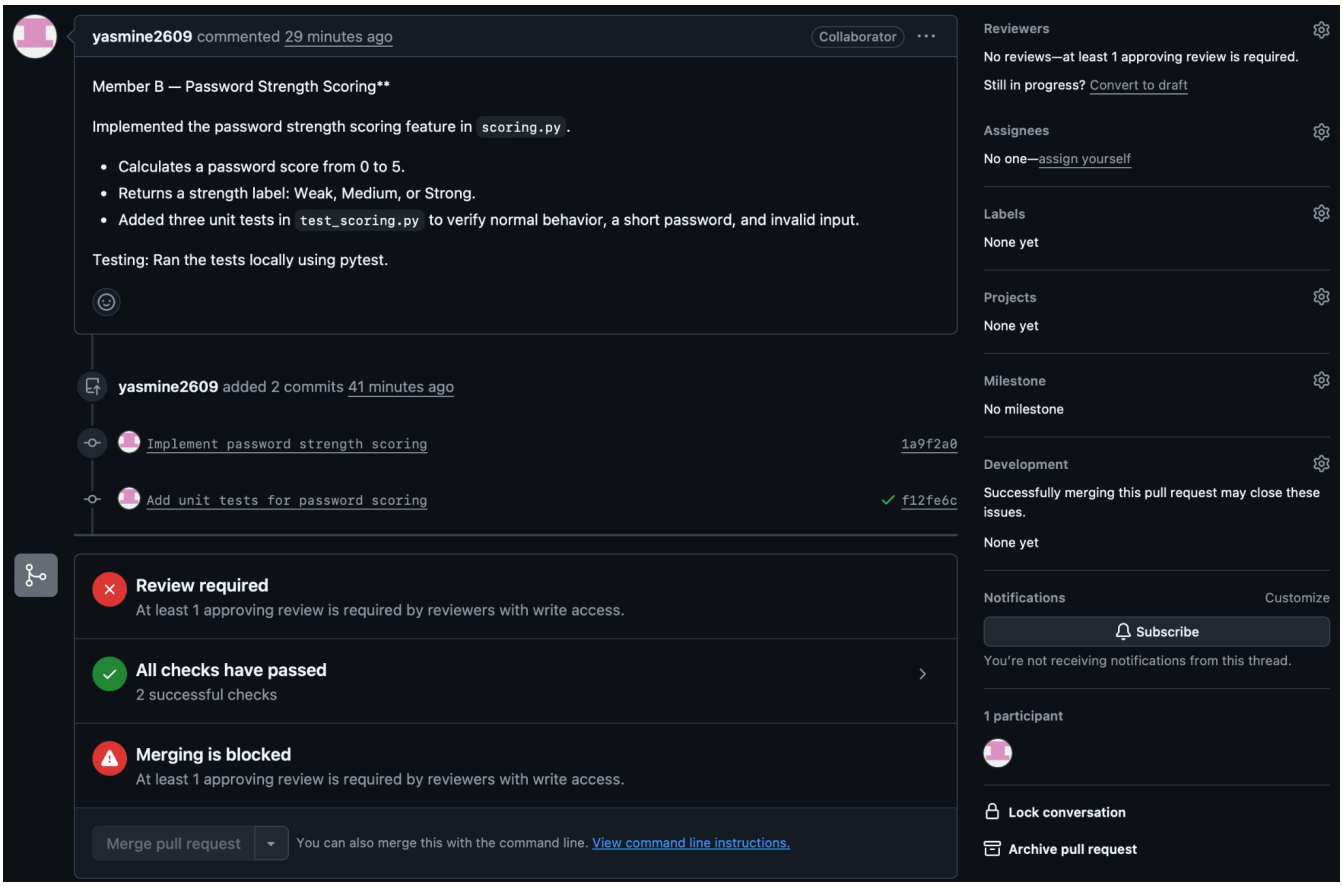


Figure 2. Scoring pull request: successful checks, but merging blocked pending an approving review.

6. Responding to review feedback

After review, Yasmine extended **tests/test_scoring.py** on the same feature branch with a test for the Weak label. The added assertion checks that **check_strength("abc")** returns **(1, "Weak")**. The update complemented the existing strong-password, short-password, and invalid-input tests.

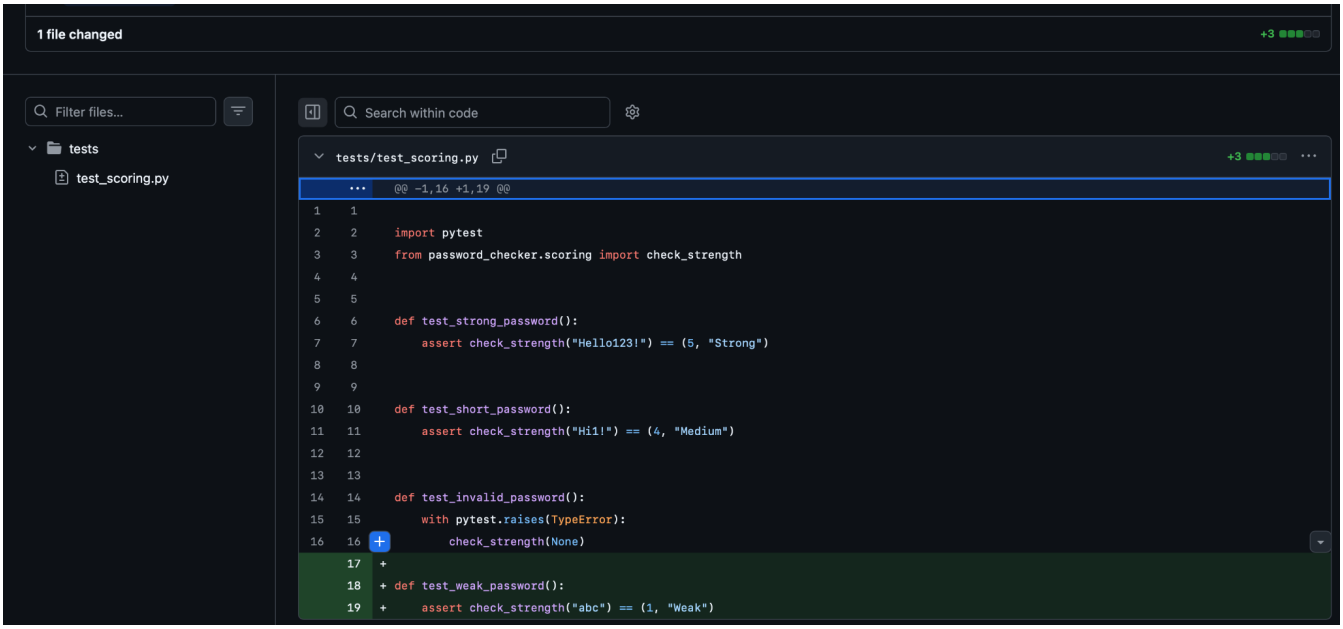


Figure 3. Additional Weak-password unit test added in response to pull-request feedback.

7. Testing the integrated project

The team ran the test suite with **pytest**. The captured terminal output reports **11 passed**, including tests for password patterns, rule checks, scoring, and setup. This confirms the test suite passed in the environment shown in the screenshot.

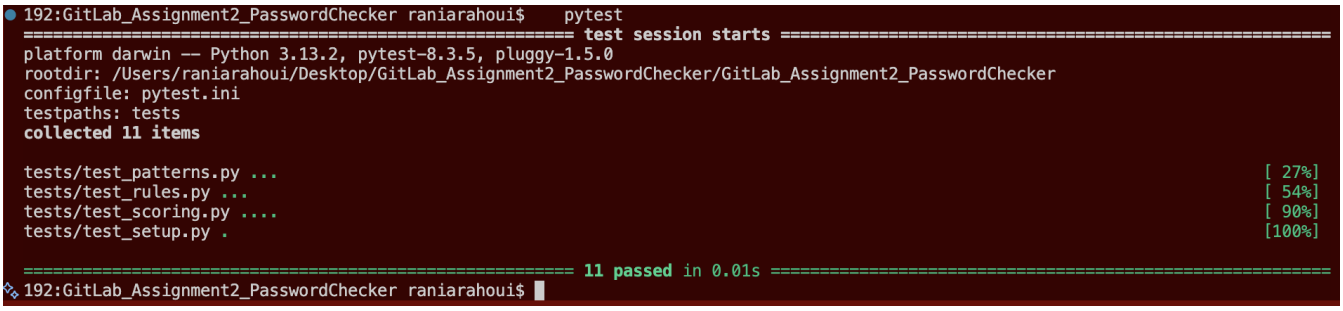


Figure 4. Local pytest output showing all 11 tests passing.

8. Continuous integration: failure and recovery

GitHub Actions provided automated checks on commits and pull requests. Kamelia deliberately introduced a failing test in a commit labeled **Trigger Test Failure for CI**, then corrected it in **Fix test and pass CI**. The commit history displays the failed status followed by successful checks, illustrating how CI helps detect and validate fixes.

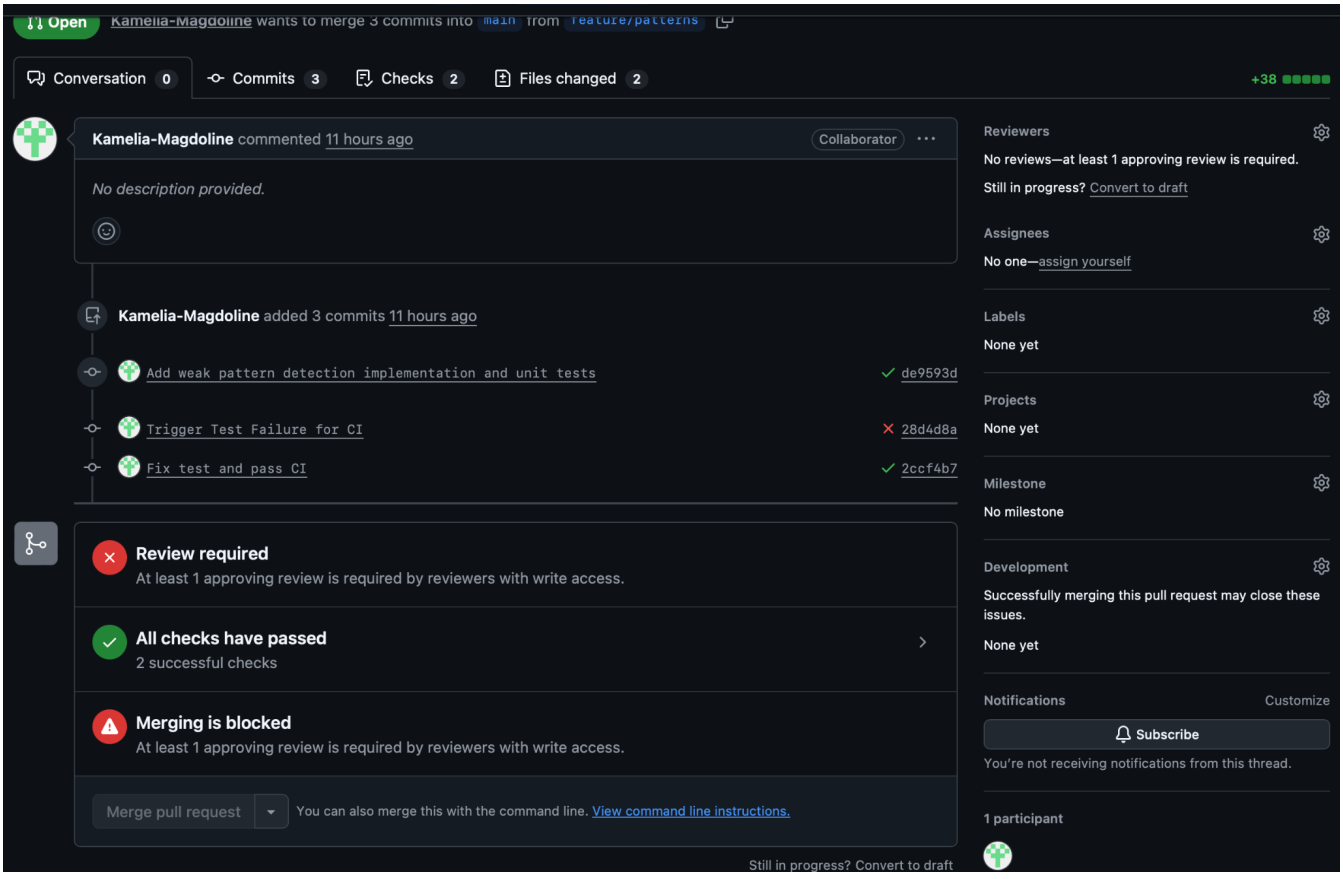


Figure 5. Pattern-detection PR history showing a failed CI commit followed by a passing correction.

9. Merge conflict and resolution

The group performed a deliberate README conflict exercise. Rania updated the project-status line in the **status-a** branch and merged it into main. Yasmine independently changed the same line in **status-b**. When main was merged into status-b, Git reported a conflict in README.md. Yasmine edited the file to keep the intended combined status, removed the conflict markers, staged the resolved README, committed the resolution, pushed status-b, and opened PR #5. After peer approval, the pull request was merged into main.

Resolution workflow: edit README.md → remove conflict markers → `git add README.md` → `git commit -m "Resolve README merge conflict"` → `git push origin status-b` → review and merge PR #5.

10. GitHub Pages deployment

The team configured GitHub Pages to publish the project documentation. The Actions view shows three successful **pages build and deployment** runs on main. The screenshots below document the page before its content was completed and the updated published page with the project theme, team, features, and instructions for running tests.

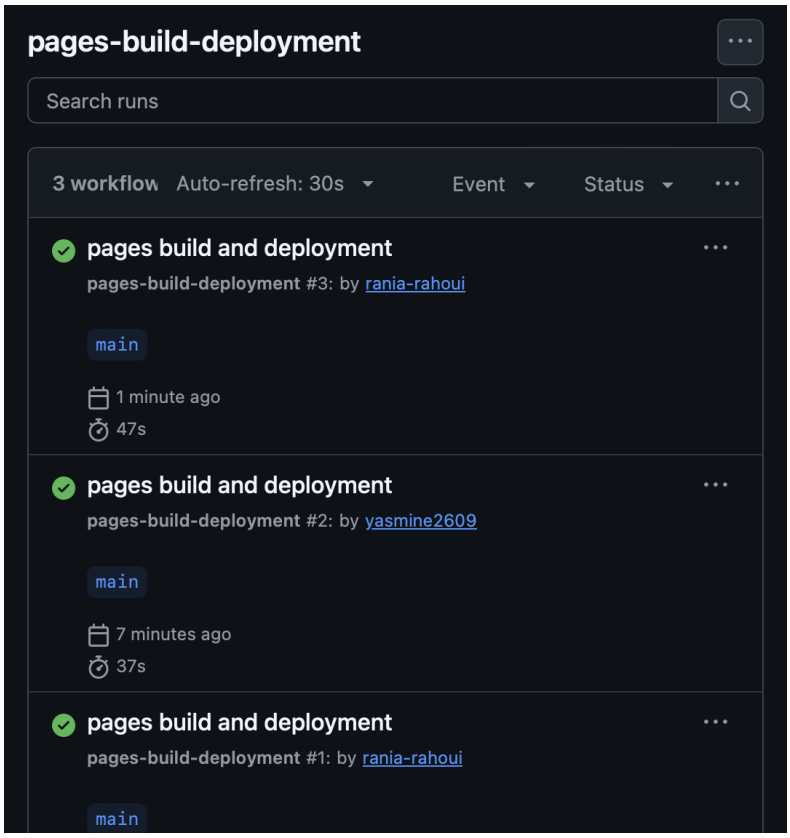


Figure 6. Successful GitHub Pages build-and-deployment workflow runs.

11. Published project page: before and after

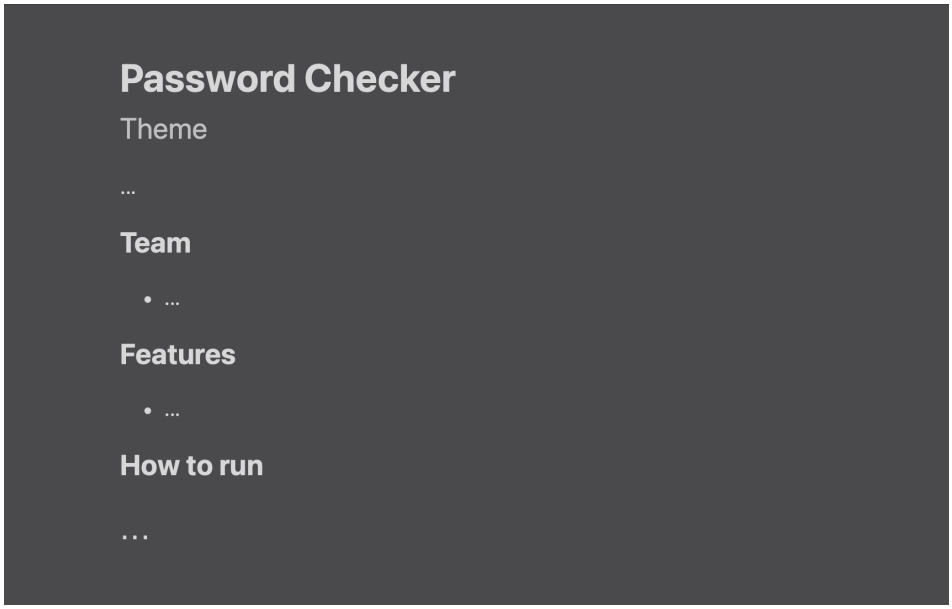


Figure 7. Earlier project page with placeholder content.

Password Checker

Theme

Password Checker: checks a password against basic rules, gives it a strength score, and flags weak patterns.

Team

- Rania Rahoui (@rania-rahoui): Rule checks
- Yasmine Boutchiche (@yasmine2609): Strength score
- Kamelia Magdoline (@Kamelia-Magdoline): Weak pattern detection

Features

- Rule checks: length of at least 8, uppercase, lowercase, digit, special character
- Strength score from 0 to 5 with a label (Weak, Medium, Strong)
- Weak pattern detection: common passwords, repeated characters, sequences

How to run

```
pip install -r requirements.txt
pytest
```

Figure 8. Updated project page listing the project purpose, all three members, features, and run commands.

12. Final results and lessons learned

The group integrated three complementary features: rule checking, password strength scoring, and weak-pattern detection. The complete local test run shown in the evidence passed all 11 tests. Pull requests required an approving review, and the team practiced responding to feedback, fixing a CI failure, and resolving a real Git merge conflict. The project documentation was deployed through GitHub Pages.

The assignment demonstrated that GitHub collaboration involves more than writing code: branches isolate changes, commits preserve history, reviews improve quality, CI checks catch problems, and merge conflict resolution allows concurrent work to be integrated safely.

13. Evidence summary

Evidence	Screenshot
Final repository and deployment	Figure 1
Review approval requirement	Figure 2
Review feedback and added test	Figure 3
11 successful local tests	Figure 4
CI failure and fix	Figure 5
Successful Pages deployments	Figure 6
Pages content before and after	Figures 7–8

Project repository: https://github.com/rania-rahoui/GitLab_Assignment2_PasswordChecker